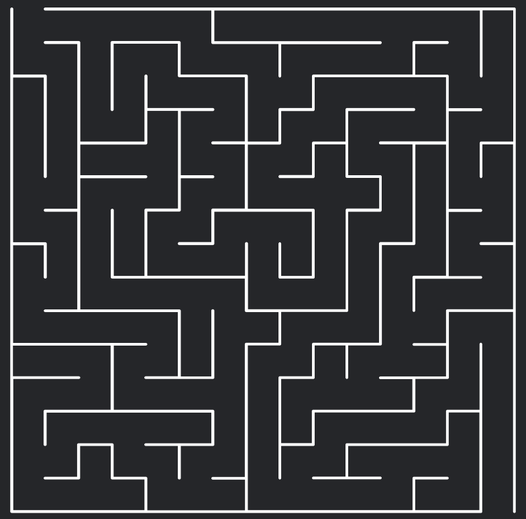
Algoritmos de Navegação

Por: Rodrigo Haruna e Rodrigo de Moraes



Problema:

Como percorrer um mapa de maneira **eficiente**?



- Não bater em coisas e travar
- Pegar vítimas

Solução incial:

Seguidor de Parede



Se manter rente a uma parede sempre, mas sem colidir

Seguidor de Parede

Há parede na minha frente?

Há parede ao meu lado?

Vai pra frente

Gira pro outro lado e anda

Gira para o lado e anda

Problema:

Como percorrer um mapa de maneira **eficiente**?



- Não bater em coisas e travar
- Pegar vítimas
- Garantir exploração total do mapa

Problema mais básico:

Como representar um mapa?

Seguidor de Parede Melhorado

Há parede ou tile visitado na minha frente?

Há parede ou tile visitado ao meu lado?

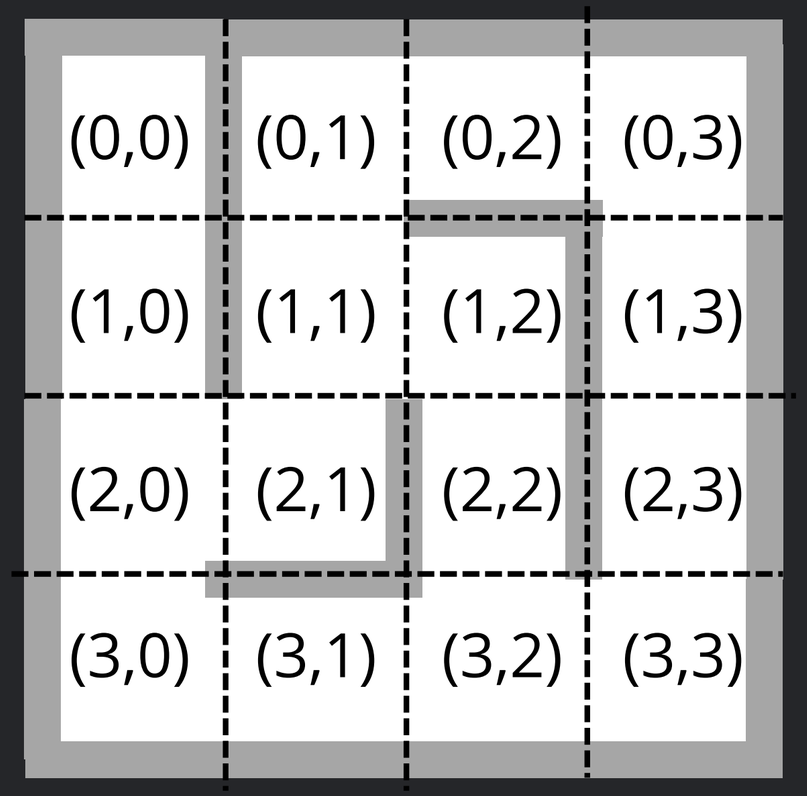
Vai para frente

Gira pro outro lado

Gira para o lado e anda

(MAZE)

Seguidor de Parede Melhorado

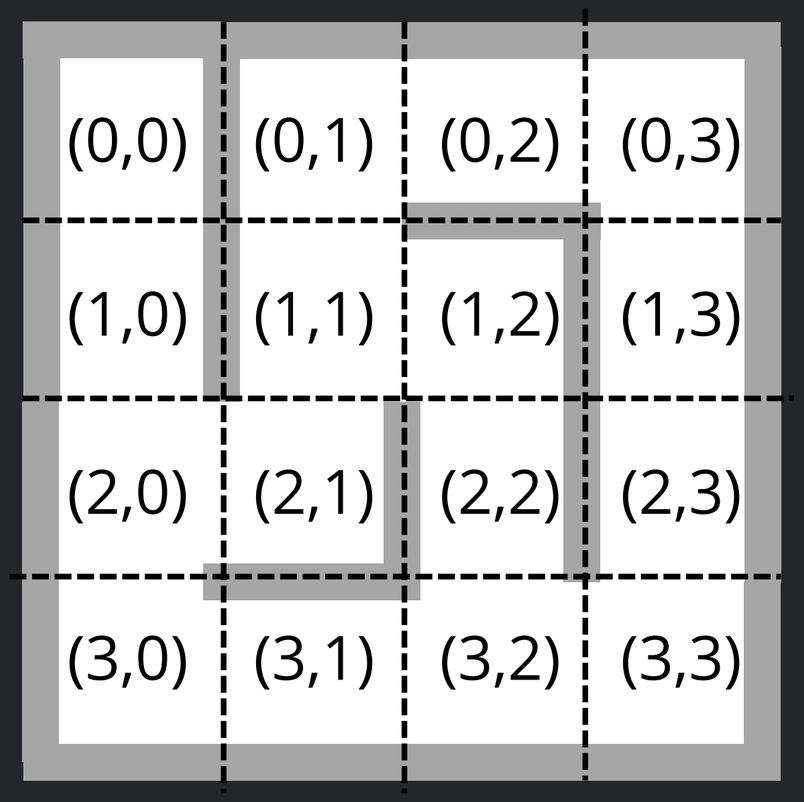


- Ideia: evitar não apenas avançar em paredes, mas também avançar em tiles já visitados

(MAZE)

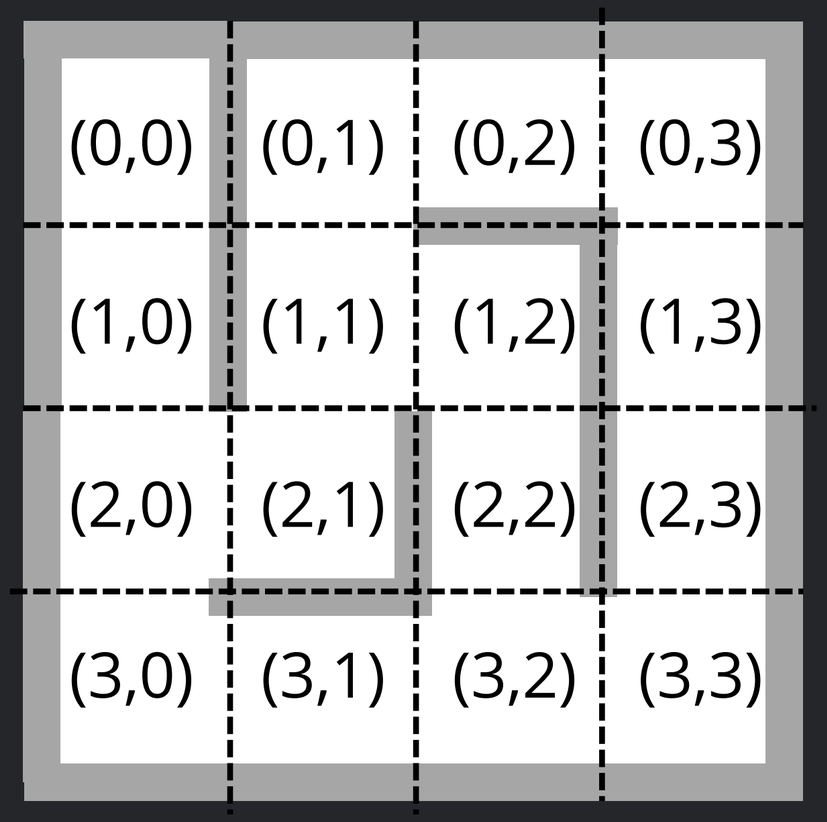
Como representar um mapa?

- Dividir em regiões: cada tile tem uma coordenada relativa ao início



(MAZE)

- Vantagem: possível acompanhar posição atual do robô



pos = (0,0)

(MAZE)

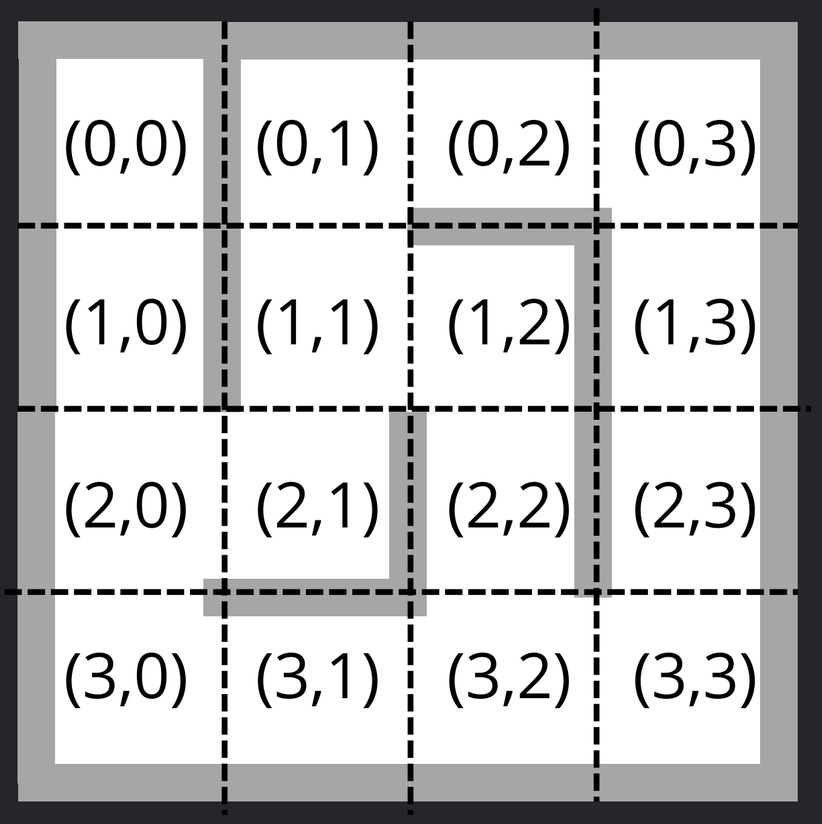
Como lidar com movimentações?

- Necessário acompanhar **orientação**

orientacao = {(1,0), (0,-1), (-1, 0), (0, 1)}

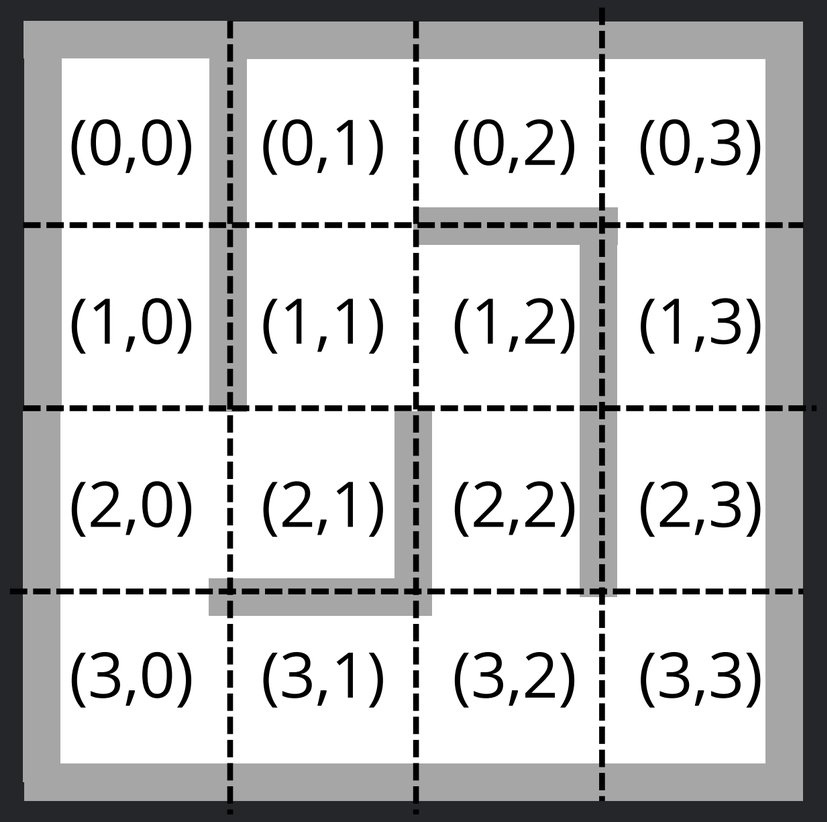
ou

orientacao = N, S, L, O



(MAZE)

- Vantagem: possível armazenar tiles já visitados em uma lista de coordenadas

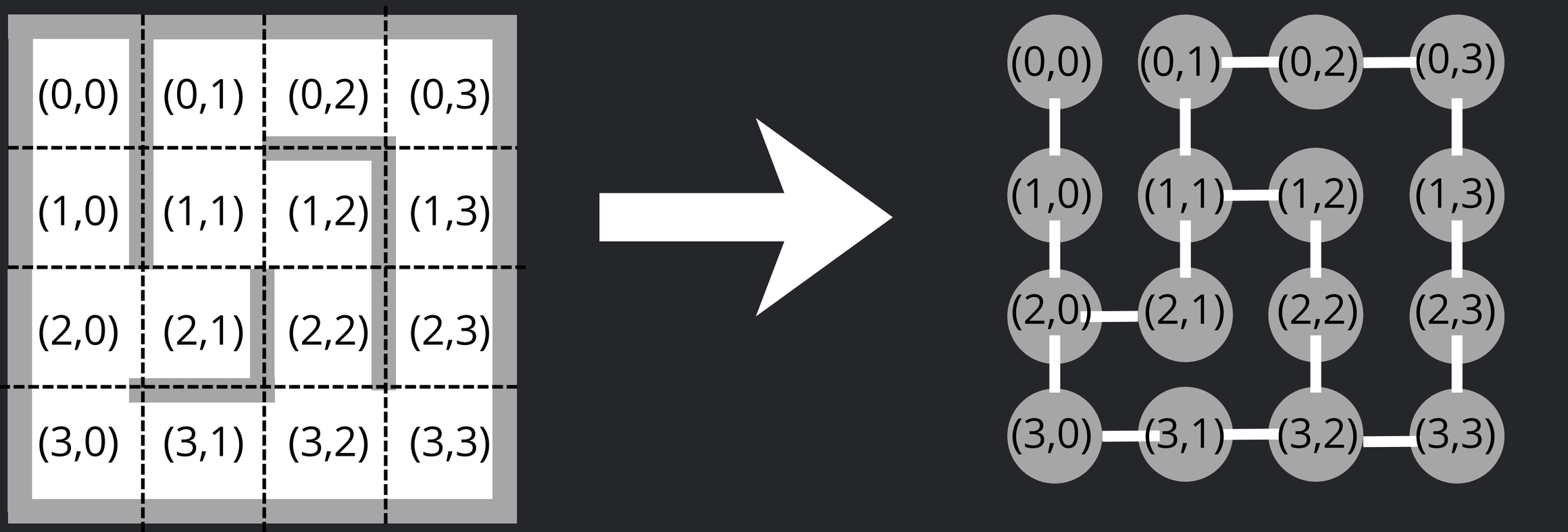


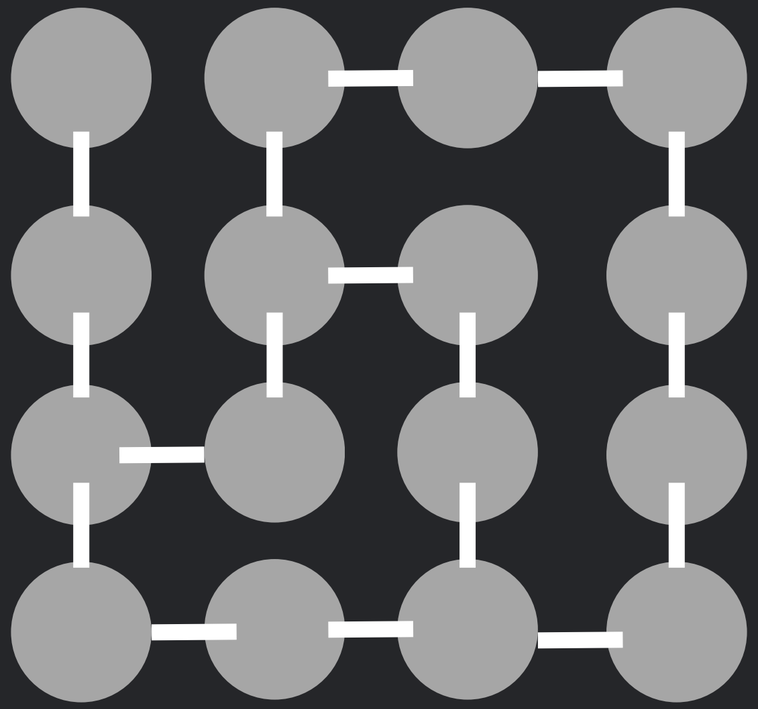
visitados = {(0,0), (1,0), (2,0), (2,1)}

(MAZE)

Como representar um mapa?

- Representar paredes e obstáculos
- Dividir em pequenas regiões

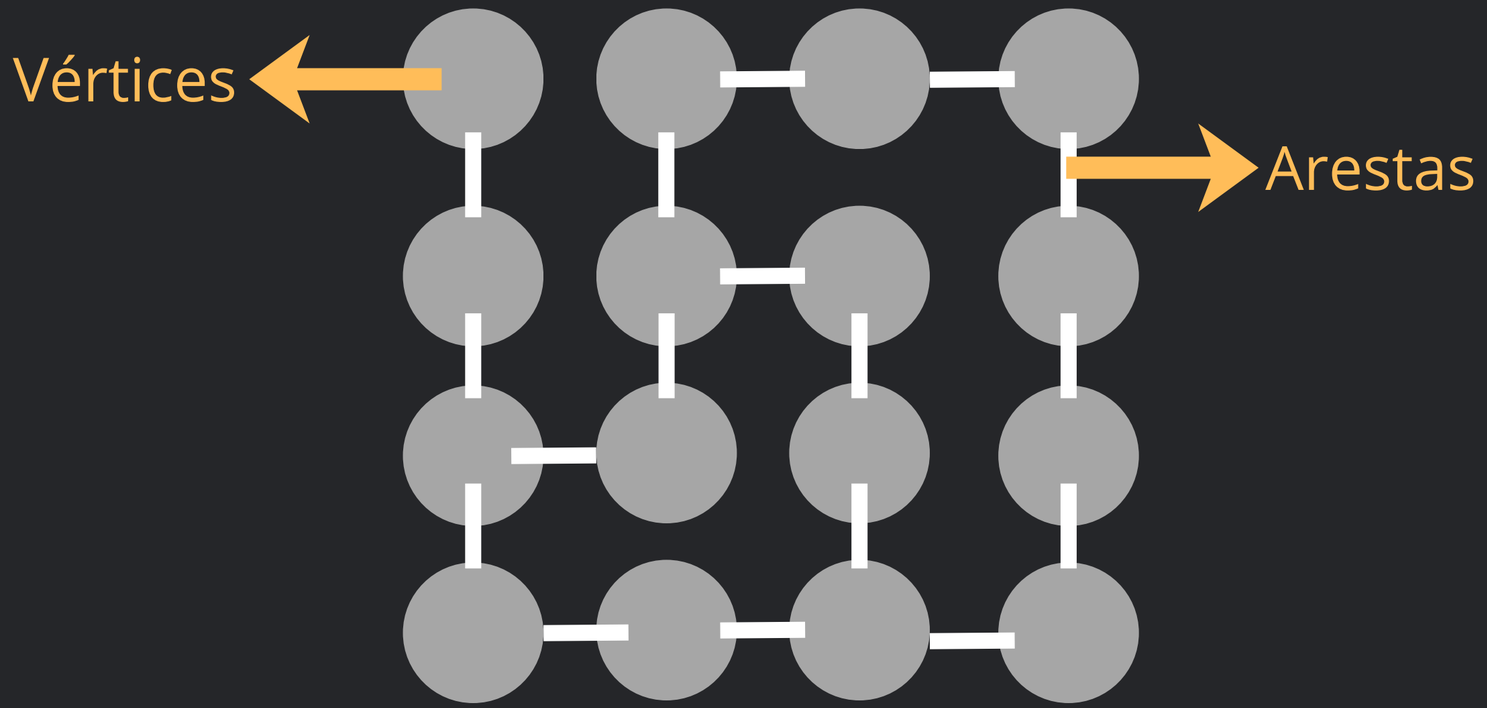




GRAFO - estrutura muito estudada!

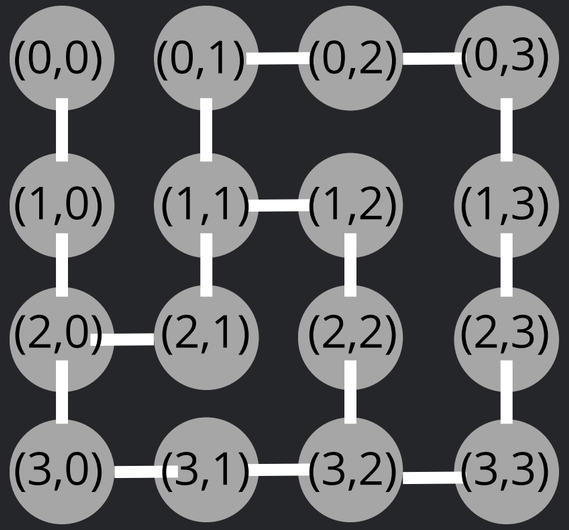
Grafos

- Definidos por vértices e arestas

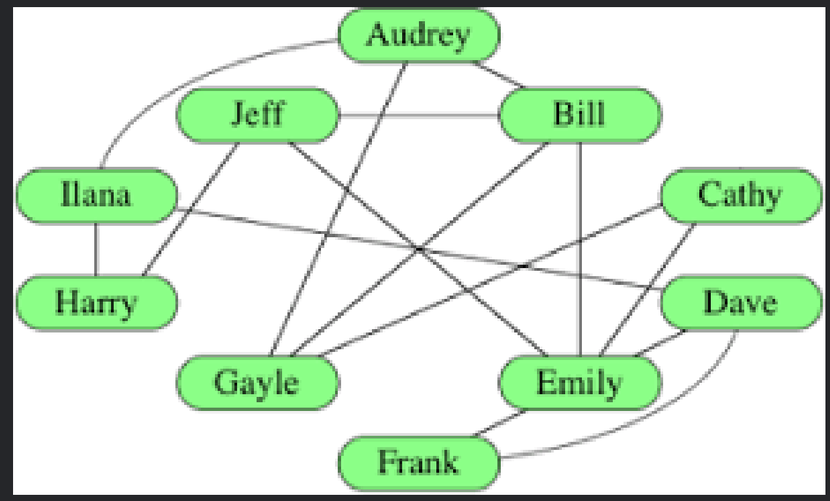


Grafos

- Expressam relações entre coisas





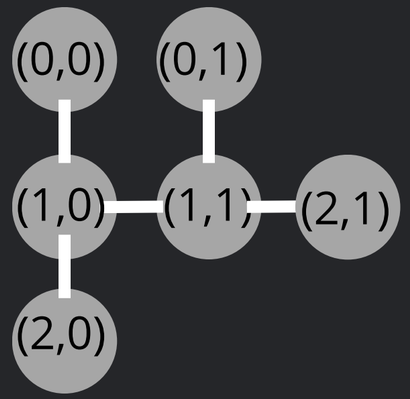


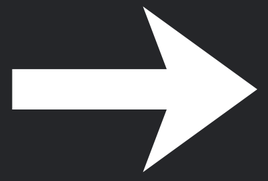
Grafos

- Possíveis representações:
  - Matriz de adjacência
  - **Lista de adjacência**

Classe Vértice

- Nome
- Vizinhos





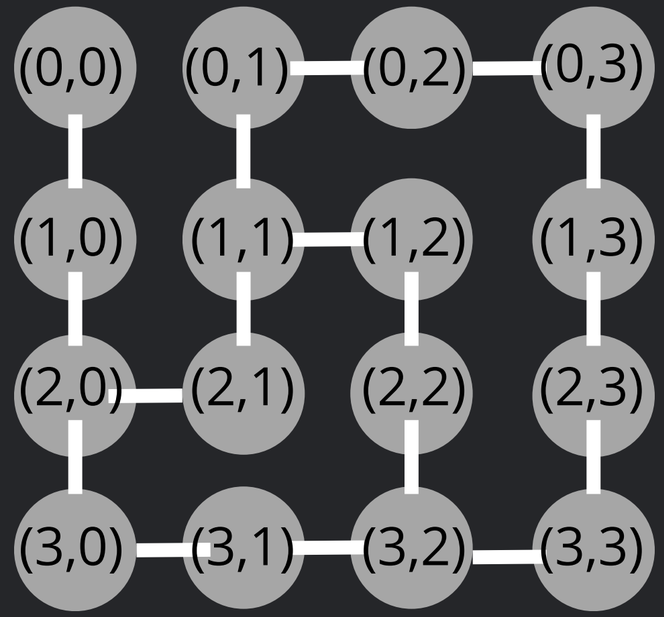
Vértices = (0,0) , (1,0) , (2,0) , (0,1) , (1,1) , (2,1)

(1,0) (1,1) (2,0) (0,0) (1,0) (1,1) (1,0) (2,1) (0,1) (1,1)

(SIMULATION)

Grafos

- Possíveis representações gerais:
  - Matriz de adjacência
  - Lista de adjacência
- No nosso caso:
  - Matriz



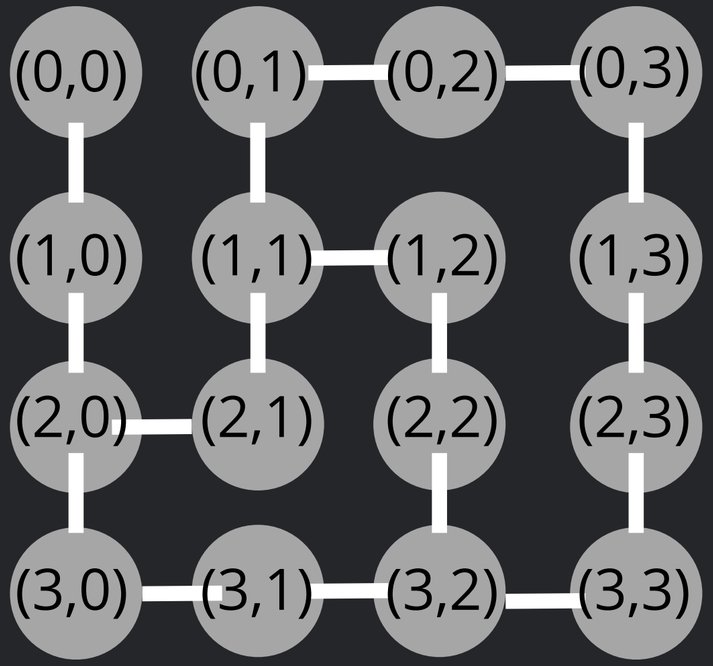
(MAZE)

Como percorrer um **mapa?**



Como percorrer um **grafo?**

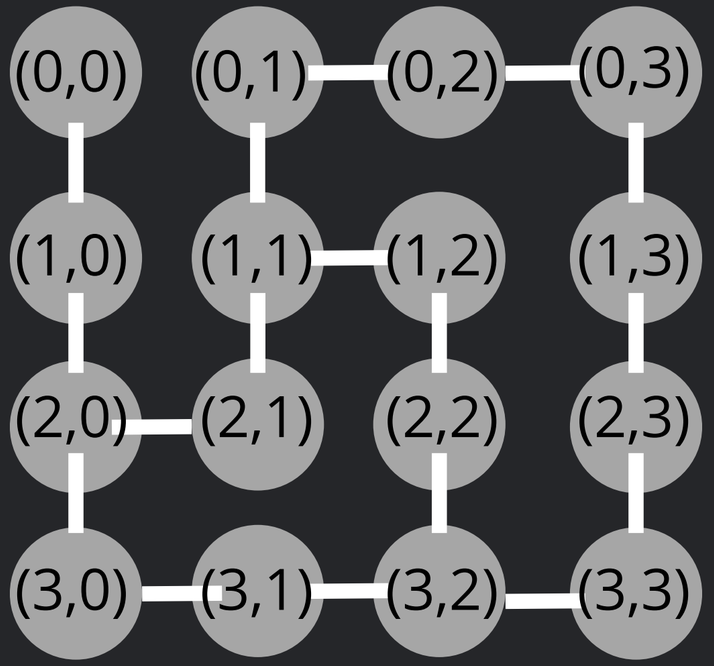
BFS
(Breadth-First Search ou Busca em Largura)



- Ideia: checar todos os vértices vizinhos antes de ir pra próxima “camada”
- FILA

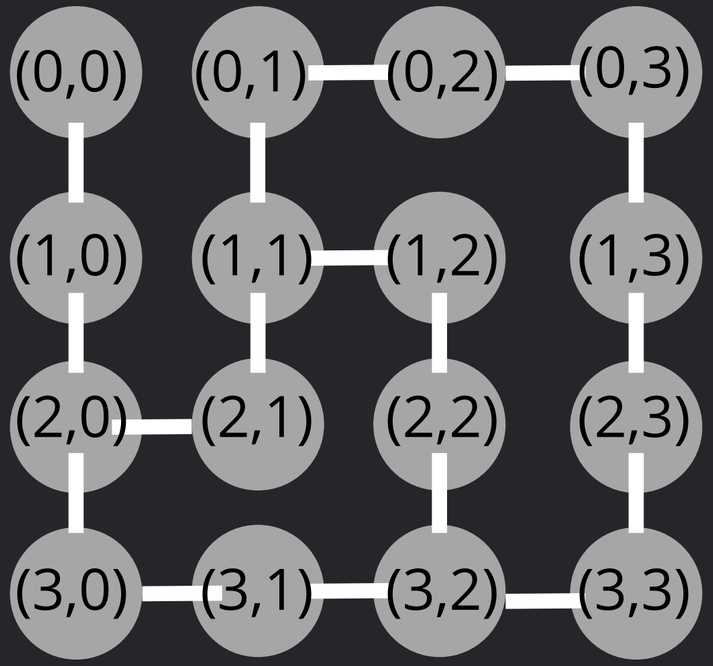
BFS

- Problema: ficar indo e voltando gasta muito tempo!!



DFS

(Depth-First Search ou Busca em Profundidade)



- Ideia: ir avançando vértice a vértice até não dar mais; então, ir voltando
- PILHA

DFS

- Detalhes de implementação

(como implementar o algoritmo puro em c)

(MAZE)

DFS

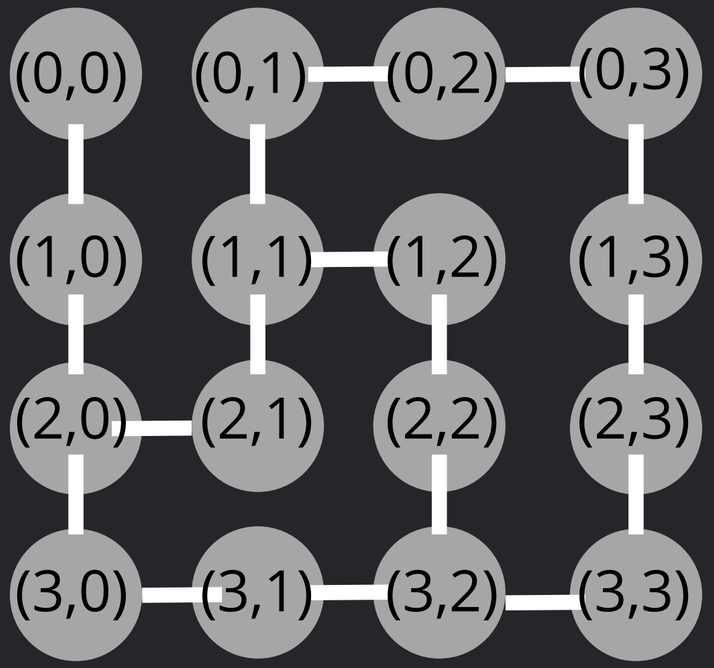
- Detalhes de implementação

(coisas a se atentar na implementação aplicada à robótica)

(MAZE)

DFS

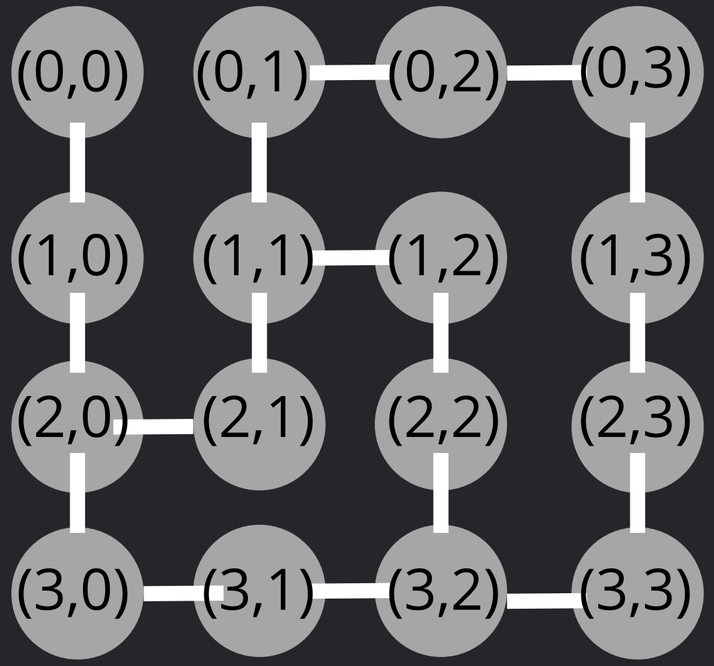
- Problema: backtracking não garante melhor caminho!!



(SIMULATION)

“DFS” + BFS

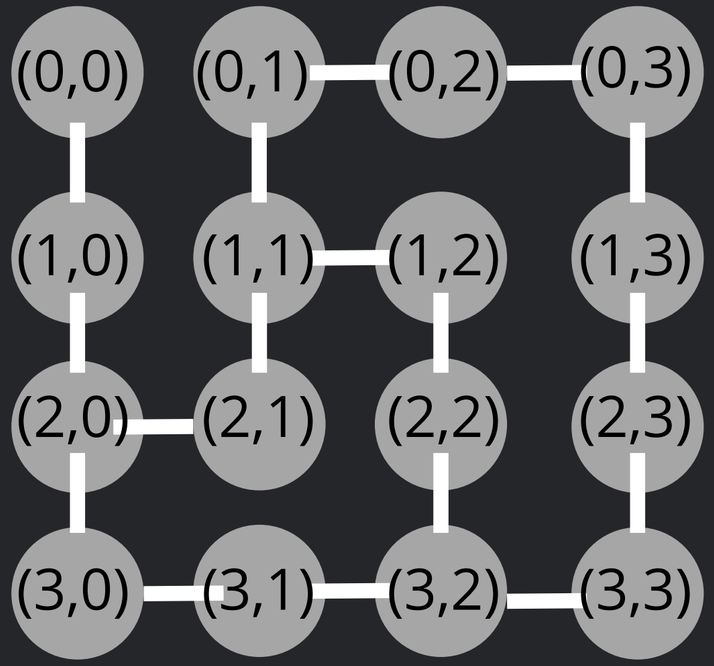
- Aproveitar da exploração da DFS
- BFS no backtracking para encontrar caminho mais rápido



(SIMULATION)

“DFS” + BFS

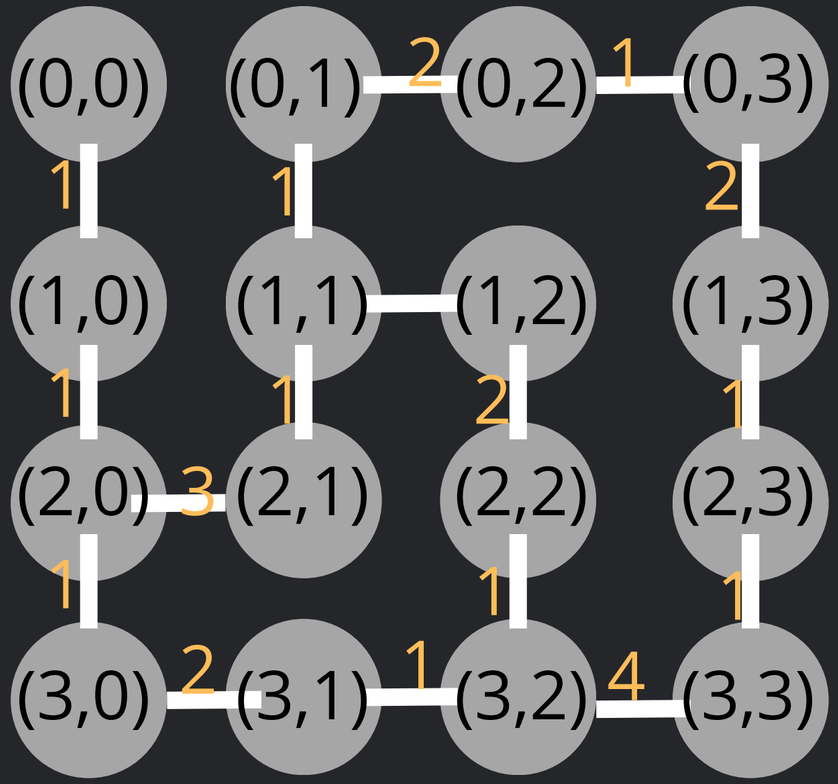
- Desvantagem: assume todos os caminhos como iguais



(SIMULATION)

“DFS” + DIJKSTRA

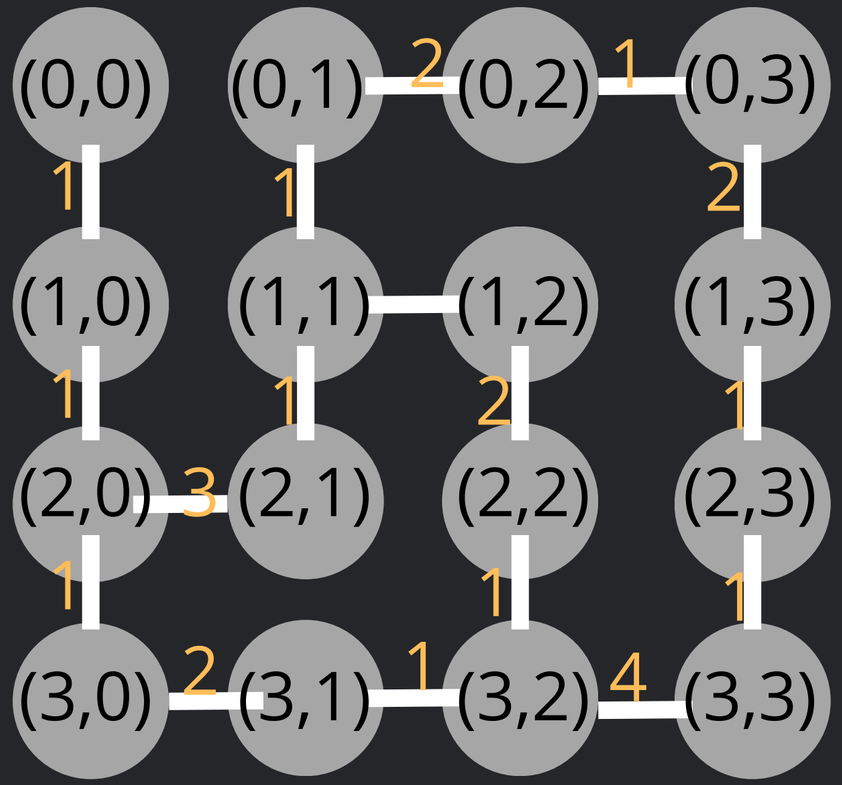
- Caminho mais rápido **com peso**
- Desvantagem: complexidade



(SIMULATION)

“DFS” + A*

- Melhor ou igual ao dijkstra em complexidade
- Desvantagem: depende de uma boa heurística



h(v) = dist(v, v0)

(SIMULATION)